Product Requirements Document (PRD)

Smart Parking Management System

Version: 1.0
Project Type: C++ Software Application
Interface: Console-based
Primary Goal: Build a functional parking management system using object-oriented software design.

---

1. Project Overview

The Smart Parking Management System is an application designed to automate the operations of a multi-floor parking facility.

The system must manage vehicle entry and exit, parking space allocation, ticket generation, parking fee calculation, payment processing, and parking availability.

It should operate as a simulation of a real-world commercial parking facility.

1.1 Project Objectives

- Eliminate manual tracking of parking spaces.
- Prevent multiple vehicles from occupying the same parking space.
- Allocate appropriate parking spaces according to vehicle categories.
- Maintain records of vehicle parking sessions.
- Calculate parking charges accurately.
- Maintain financial and operational records.
- Handle invalid operations without crashing.

2. System Scope

2.1 Supported Vehicles

The system must support three vehicle categories:

Vehicle	Description
Motorcycle	Two-wheeled motor vehicles
Car	Standard passenger cars
Truck	Large commercial vehicles

Every vehicle entering the facility must have a unique registration number identifying it.

2.2 Parking Facility

The facility must support multiple floors.

Each floor contains a configurable number of parking spaces.

Parking spaces are categorized as:

- Motorcycle spaces
- Car spaces
- Truck spaces

For Version 1.0, each parking space can accommodate only its corresponding vehicle category.

Every parking space must have a unique identifier within the facility.

3. Functional Requirements

FR-01: Parking Facility Configuration

The system must allow an administrator to:

1. Create a parking facility.
2. Specify the total number of floors.
3. Configure the parking spaces available on each floor.
4. Assign supported vehicle categories to spaces.

5. View the current facility configuration.

Business rules:

- A parking facility must contain at least one floor.
- Each floor must contain at least one parking space.
- Parking space identifiers must not be duplicated.
- Configuration changes must not invalidate active parking sessions.

FR-02: Vehicle Entry

The system must allow a vehicle to enter the parking facility.

Required input:

- Vehicle registration number
- Vehicle category
- Entry date and time

Expected behavior:

1. Check whether the vehicle is already parked.
2. Check whether a compatible parking space is available.
3. Assign a suitable parking space.
4. Mark the allocated space as occupied.
5. Generate a parking ticket.
6. Display the entry confirmation.

Business rules:

- A vehicle cannot have two active parking sessions simultaneously.
- Entry must be rejected when no compatible parking space is available.
- An unsuccessful entry must not change parking availability.
- Vehicle registration numbers must not be empty.

FR-03: Parking Space Allocation

The system must automatically allocate parking spaces.

Allocation policy for Version 1.0:

- Prefer the lowest-numbered floor containing an available compatible space.
- Within that floor, select the space with the lowest identifier.
- If no compatible space exists, reject the entry request.

Expected behavior:

The system must accurately maintain the occupancy status of each parking space.

FR-04: Parking Ticket Management

A parking ticket must be generated for every successful vehicle entry.

Each ticket must contain:

Field	Description
Ticket ID	Unique identification number
Registration Number	Vehicle identification
Vehicle Category	Motorcycle, Car, or Truck
Floor Number	Allocated floor
Parking Space ID	Allocated space
Entry Time	Date and time of entry
Ticket Status	Active or Closed

Business rules:

- Each parking session must have exactly one ticket.
- Tickets cannot be reused for a different parking session.
- Closed tickets cannot be used to initiate another exit.
- Historical tickets must remain accessible.

FR-05: Vehicle Exit

The system must support vehicle exit using a ticket ID.

Expected behavior:

1. Validate the ticket.
2. Retrieve the corresponding active parking session.
3. Calculate the total parking duration.
4. Calculate the amount payable.
5. Display the bill.

6. Process payment.
7. Mark the ticket as closed after successful payment.
8. Release the occupied parking space.

Business rules:

- Invalid ticket IDs must be rejected.
- An already closed ticket cannot be processed again.
- Exit time cannot precede entry time.
- A vehicle must not be marked as exited until payment succeeds.
- Parking availability must increase after a successful exit.

FR-06: Parking Fee Calculation

Parking fees depend on vehicle category and total parking duration.

Base pricing:

Vehicle	Rate per hour
Motorcycle	₹10
Car	₹20
Truck	₹30

Billing rules:

- The minimum billable duration is one hour.
- Partial hours are rounded up to the next full hour.
- A vehicle parked for 1 hour and 10 minutes is charged for 2 hours.
- A vehicle parked for exactly 2 hours is charged for 2 hours.
- Rates are configured for the facility and apply to all floors.

Example:

Car parked for 2 hours 20 minutes:

Billable duration = 3 hours
Total = 3 × ₹20 = ₹60

The system must be capable of supporting changes to pricing rules in future versions.

FR-07: Payment Processing

The system must support the following simulated payment methods:

- Cash
- UPI
- Card

No actual banking or payment gateway integration is required.

Expected behavior:

- Display the amount payable.
- Accept a payment method.
- Record whether payment succeeded or failed.
- Generate a payment record for successful transactions.

Business rules:

- A failed payment must not release the parking space.
- A successful transaction must not be charged twice.
- Each successful transaction must have a unique transaction ID.
- Payment records must remain available after vehicle exit.

FR-08: Parking Availability

The system must display parking availability.

Required information:

- Total parking capacity
- Total occupied spaces
- Total available spaces
- Floor-wise availability
- Vehicle-category-wise availability

Example display:

```
PARKING STATUS

Floor 1
Motorcycle: 4/10 available
Car:        8/20 available
Truck:      2/5 available

Floor 2
Motorcycle: 7/10 available
Car:       12/20 available
Truck:      0/5 available
```

The counts must reflect the latest successful entry and exit operations.

FR-09: Search and Vehicle Tracking

The system must support searching for:

1. An active vehicle using its registration number.
2. A parking session using its ticket ID.
3. All currently parked vehicles.
4. Completed parking sessions.

For an active vehicle, the system must display its registration number, category, allocated space, floor, entry time, and ticket ID.

Searching for an unknown vehicle or ticket must produce an appropriate message.

FR-10: Reports and Statistics

The system must provide the following reports:

Occupancy Report

- Total capacity
- Occupied spaces
- Available spaces
- Occupancy percentage

Revenue Report

- Total revenue collected

- Number of successful payments
- Revenue by vehicle category

Parking Activity Report

- Total successful vehicle entries
- Total completed parking sessions
- Number of currently parked vehicles

Reports must use actual recorded system data rather than hardcoded values.

FR-11: Data Persistence

The system must preserve important information when the application closes.

After restarting, it must recover:

- Parking facility configuration
- Current parking occupancy
- Active tickets
- Historical parking sessions
- Completed payment records
- Pricing configuration

Business rules:

- Previously closed tickets must remain closed.
- Currently occupied spaces must remain occupied.
- Ticket and transaction identifiers must not collide with historical identifiers.
- Missing or corrupted saved data must not cause an uncontrolled application crash.

The persistence format and storage mechanism are implementation decisions.

4. User Interface Requirements

The application must provide a console-based interface.

At minimum, users must be able to perform the following actions:

1. Configure the parking facility.
2. Register a vehicle entry.
3. Process a vehicle exit and payment.
4. View parking availability.

5. Search for vehicles or tickets.
6. View reports.
7. View parking history.
8. Exit the application.

The interface must display clear success and error messages.

Invalid menu selections or input values must be handled without terminating the application unexpectedly.

Authentication and multiple user accounts are outside the scope of Version 1.0.

5. Non-Functional Requirements

NFR-01: Reliability

The application must not crash during normal usage or expected invalid inputs.

NFR-02: Data Consistency

At any point:

- One parking space can have at most one active vehicle.
- One vehicle can have at most one active parking session.
- Active tickets must correspond to occupied spaces.
- Available spaces + occupied spaces = total configured capacity.
- Failed operations must not leave partially updated records.

NFR-03: Maintainability

The project should be organized so that future requirements can be introduced without rewriting the entire application.

NFR-04: Extensibility

The system should be reasonably adaptable to future features such as:

- Additional vehicle categories
- Alternative parking allocation policies
- New pricing schemes
- New payment methods

- Multiple parking facilities

NFR-05: Testability

Core operations must be verifiable independently of manually navigating the entire console interface.

NFR-06: Performance

For Version 1.0, the system should function correctly with:

- Up to 10 floors
- Up to 100 parking spaces per floor
- At least 1,000 historical parking sessions

Real-time distributed operation is not required.

6. Edge Cases and Error Handling

The following scenarios must be handled correctly:

Scenario	Expected Result
Vehicle already parked attempts entry	Reject duplicate entry
No compatible parking space available	Reject entry
Invalid vehicle category	Reject input
Invalid or empty registration number	Reject input
Unknown ticket ID	Display error
Closed ticket submitted for exit	Reject duplicate exit
Exit time earlier than entry	Reject invalid operation
Zero-minute parking duration	Charge minimum one hour
Payment fails	Keep session active
Same ticket is submitted twice for payment	Prevent duplicate successful charge
Last available space is occupied	Availability becomes zero

Scenario	Expected Result
Vehicle exits a full floor	One space becomes available
Application restarts with active sessions	Restore sessions correctly
Invalid administrative configuration	Reject invalid change

7. Acceptance Criteria

The project is considered functionally complete when all the following conditions are satisfied:

- [ ] A parking facility with multiple floors can be configured.
- [ ] Different vehicle categories are supported.
- [ ] Vehicles receive appropriate available spaces.
- [ ] Duplicate active entries are prevented.
- [ ] Successful entries generate unique tickets.
- [ ] Parking fees are calculated correctly.
- [ ] Multiple simulated payment methods work.
- [ ] Failed payments do not complete exits.
- [ ] Successful exits release parking spaces.
- [ ] Parking availability remains accurate.
- [ ] Users can search active and historical sessions.
- [ ] Reports display accurate operational and revenue data.
- [ ] Application data survives restarts.
- [ ] Invalid operations are handled gracefully.
- [ ] The complete system can be operated through a console interface.

8. Out of Scope — Version 1.0

The following are not required:

- Graphical user interface
- Website or mobile application
- Actual payment gateway integration
- License plate recognition
- Physical parking sensors
- QR code scanning

- Reservation or advance booking
- Multiple simultaneous operators
- Networked or distributed deployment
- Dynamic pricing based on demand
- Authentication and role-based access control

9. Future Enhancements — Version 2.0

Potential future requirements include:

- Reserved and VIP parking spaces
- Electric vehicle charging spots
- Discount coupons and subscriptions
- Multiple parking allocation strategies
- Weekend, peak-hour, and event-based pricing
- Advance parking reservations
- Automatic entry using number plate recognition
- Multiple facilities with centralized management
- Concurrent entry and exit processing

These features should not be implemented until Version 1.0 is complete.

10. Final Deliverables

The completed project must contain:

1. A functional C++ console application.
2. Source code organized into appropriate files.
3. Instructions for compiling and running the application.
4. A README describing functionality and user workflows.
5. Test cases covering core functionality and important edge cases.
6. Sample input and output demonstrating successful and unsuccessful operations.

Project completion condition: A user must be able to configure a multi-floor parking facility, park different types of vehicles, receive tickets, calculate and pay parking charges, exit vehicles, inspect availability and reports, and restart the application without losing important records.

End of PRD — Version 1.0